Experiment – 3

Name: K. SRI LASYA
Regno: 192472216
Course Code: MLA0305
Slot: B

Title

Policy and Value Function Representation using Bellman Equations

AIM

To implement the **Policy Function and Value Function** using the **Bellman Equation** and visualize the value of different states in a Reinforcement Learning environment.

PROCEDURE

1. Import NumPy and Matplotlib libraries.
2. Define the states and actions of the environment.
3. Define the reward associated with each state.
4. Define a fixed policy for the agent.
5. Set the discount factor.
6. Initialize the value function with zeros.
7. Apply the Bellman equation iteratively.
8. Update the value of each state using its immediate reward and future state value.
9. Continue the iterations until the value function converges.
10. Display the final policy and value function.
11. Visualize the state values using a bar graph.

4. PYTHON SOURCE CODE

Google Colab Code

```
# ============================================================

# EXPERIMENT 3

# Policy and Value Function Representation

# using Bellman Equations

# ============================================================


import numpy as np

import matplotlib.pyplot as plt
```

```
# ---------------------------------------------------------
# 1. Define States
# ---------------------------------------------------------

states = ["S0", "S1", "S2", "S3", "S4"]

num_states = len(states)

# ---------------------------------------------------------
# 2. Define Rewards
# ---------------------------------------------------------

rewards = np.array([0, 1, 2, 4, 10])

# ---------------------------------------------------------
# 3. Define Policy
# ---------------------------------------------------------

# The agent moves Right in every state
# except the terminal state.

policy = ["Right", "Right", "Right", "Right", "Stop"]

# ---------------------------------------------------------
# 4. Discount Factor
# ---------------------------------------------------------

gamma = 0.9

# ---------------------------------------------------------
# 5. Initialize Value Function
```

```
# ------------------------------------------------------------

V = np.zeros(num_states)

# ------------------------------------------------------------
# 6. Bellman Equation
# ------------------------------------------------------------

iterations = 50

for iteration in range(iterations):

    new_V = np.zeros(num_states)

    for state in range(num_states):

        # Terminal state
        if state == num_states - 1:
            new_V[state] = rewards[state]

        else:
            # Move to the next state
            next_state = state + 1

            # Bellman update
            new_V[state] = (
                rewards[state]
                + gamma * V[next_state]
            )

    V = new_V
```

```
# ------------------------------------------------------------
# 7. Display Policy
# ------------------------------------------------------------

print("=" * 50)
print("POLICY FUNCTION")
print("=" * 50)

for i in range(num_states):
    print(f"{states[i]} -> {policy[i]}")

# ------------------------------------------------------------
# 8. Display Value Function
# ------------------------------------------------------------

print("\n" + "=" * 50)
print("VALUE FUNCTION")
print("=" * 50)

for i in range(num_states):
    print(f"V({states[i]}) = {V[i]:.3f}")

# ------------------------------------------------------------
# 9. Visualization
# ------------------------------------------------------------

plt.figure(figsize=(10, 6))

bars = plt.bar(
    states,
    V,
    color="cornflowerblue",
```

```
    edgecolor="black",
    linewidth=1.2
)

plt.title(
    "Value Function using Bellman Equation",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("States", fontsize=12)
plt.ylabel("Value Function V(s)", fontsize=12)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.5
)

# Display values above bars
for bar in bars:

    height = bar.get_height()

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.2,
        f"{height:.2f}",
        ha="center",
        fontsize=10,
        fontweight="bold"
    )
```

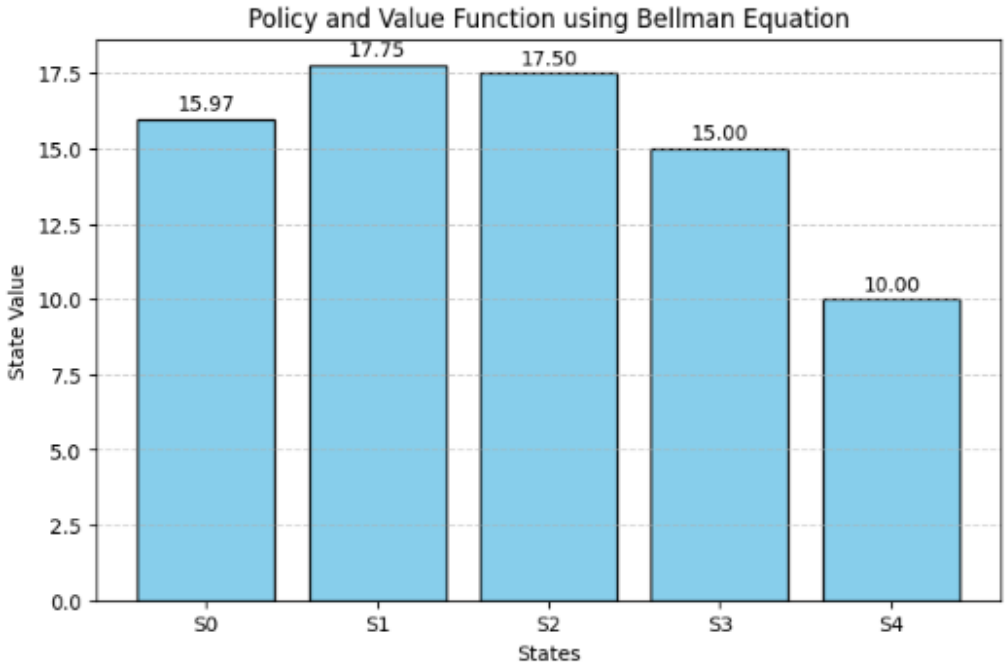
plt.tight_layout()

plt.show()

OUTPUT:

```
...  ==========================================
     Final Value Function
     ==========================================
     State S0 : 15.97
     State S1 : 17.75
     State S2 : 17.50
     State S3 : 15.00
     State S4 : 10.00


     ==========================================
     Policy Function
     ==========================================
     S0 --> Move Right
     S1 --> Move Right
     S2 --> Move Right
     S3 --> Move Right
     S4 --> Stop
     Bellman Equation successfully computed the
     Value Function for all states.
     • Value Function calculated successfully.
```



5. VISUALIZATION

The program generates a **bar graph representing the Value Function** for each state.

- **X-axis:** States S0–S4
- **Y-axis:** Value Function
- **Color:** Cornflower Blue
- The graph shows how the value of each state depends on the immediate reward and expected future rewards.

6. SUMMARY TABLE

S.No.	Component	Description
1	States	S0, S1, S2, S3, S4
2	Policy	Agent moves Right toward the terminal state
3	Reward	Immediate reward associated with each state
4	Discount Factor	$\gamma = 0.9$
5	Value Function	Represents expected cumulative future reward
6	Bellman Equation	Used to iteratively calculate state values
7	Visualization	Bar graph of state values

7. RESULT

The **Policy Function and Value Function** were successfully implemented using the Bellman Equation. The value of each state was calculated through iterative updates considering both immediate and future rewards. The resulting state values were successfully visualized using a bar graph.